

FICHES DE TRAVAUX PRATIQUES

CRYPTOGRAPHIE

Chiffrement Classique · Symétrique · Asymétrique · Hachage · Signature Numérique

Niveau : Ing 3 — Cybersécurité	Volume : 7 TPs
Langage : Python 3.9+ (facultatif)	Libs (facultatif) : cryptography · pycryptodome · hashlib · sympy
Prérequis : Arithmétique modulaire, Algorithmique	Évaluation : Rapport + Code + Vidéo démo

N°	Thème principal	Algorithmes / Outils
TP 1	Chiffrement Classique	César, Vigenère, Hill, OTP (Vernam)
TP 2	Crypto Symétrique	RC4, DES, AES-128/192/256 + 5 finalistes NIST
TP 3	Crypto Asymétrique	Diffie-Hellman, RSA, ElGamal, ECC
TP 4	Fonctions de Hachage	MD5, SHA-256, SHA-512, HMAC
TP 5	Signatures Numériques	RSA-PSS, ElGamal, DSA, ECDSA
TP 6	Application Sécurisée	Sockets TCP/UDP, Bluetooth, Wi-Fi, vote électronique

TP 1 Chiffrement Classique

Objectif : Implémenter César, Vigenère, Hill et OTP — analyser leurs faiblesses

Durée : 3 h (estimation)

Exercice 1.1 — Chiffre de César

1. **Implémenter** `chiffrer_cesar(texte, k)` et `dechiffrer_cesar(texte, k)` — ignorer les espaces et la casse.
2. **Attaque par force brute** : tester les 26 clés possibles, afficher toutes les déclinaisons et identifier automatiquement le texte français valide via un dictionnaire de mots courants.
3. **Analyse de fréquences** : calculer l'indice de coïncidence (IC) du cryptogramme, le comparer à l'IC du français ($\approx 0,074$) et déduire k sans force brute.

Exercice 1.2 — Chiffre de Vigenère

Travail demandé :

1. Implémenter `chiffrer_vigenere(texte, cle)` et `dechiffrer_vigenere(texte, cle)` ; la clé est un mot alphabétique.
2. **Test de Kasiski** : implémenter la recherche de trigrammes répétés dans le cryptogramme pour estimer la longueur probable de la clé.
3. **Analyse par IC** : pour chaque décalage k possible, découper le cryptogramme en k sous-séquences, calculer l'IC de chacune et retrouver les lettres de la clé par analyse de fréquences.
4. **Question** : En quoi la longueur de la clé impacte-t-elle la sécurité ? Que se passe-t-il quand $|K| = |M|$? (lien avec OTP)

Exercice 1.3 — Chiffre de Hill

Travail demandé :

1. Implémenter Hill 2×2 et 3×3 : chiffrement par blocs de 2 et 3 lettres, calcul de l'inverse modulaire de la matrice clé via $\det^{-1} \times \text{adj}(K) \bmod 26$, il faut vérifier la validité de la matrice.
2. **Attaque à clair connu** : implémenter l'attaque sur un exemple et récupérer la clé.
3. **Question** : Pourquoi Hill est-il vulnérable à l'attaque à clair connu même pour de grandes matrices ?

Exercice 1.4 — One-Time Pad (Vernam)

Travail demandé :

1. **Implémenter OTP** : générer une clé, chiffrer par XOR octet à octet, déchiffrer et vérifier la restitution exacte.
2. **Vulnérabilité de réutilisation** : chiffrer deux messages $M1$ et $M2$ avec la même clé K ; calculer $C1 \text{ XOR } C2 = M1 \text{ XOR } M2$ et montrer qu'un attaquant peut en extraire des informations.
3. Analyser le résultat : appliquer des statistiques de langue sur $M1 \text{ XOR } M2$ pour récupérer partiellement $M1$ et $M2$ (attaque « crib dragging »).
4. **Question** : L'OTP est théoriquement parfait mais pratiquement inutilisable, citer des obstacles concrets.

TP 2 Cryptographie Symétrique Moderne

Objectif : RC4, DES, AES et ses 5 finalistes NIST — modes opératoires et vulnérabilités

Libs (facultatif) : pycryptodome · cryptography

Exercice 2.1 — RC4 (chiffrement par flot)

Travail demandé :

1. Implémenter RC4 en deux phases : 1) KSA (permutation de S selon la clé) et 2) PRGA (génération du keystream par échanges successifs dans S).
2. **Vulnérabilité WEP** : générer les keystreams pour des IV faibles (IV commençant par 0x00, 0x01...) et observer la corrélation entre le premier octet du keystream et la clé.
3. **Biais statistiques (facultatif)** : générer 10 000 keystreams pour des clés aléatoires, histogrammer le 2e octet et observer le biais vers 0 (RC4 bias) — raison du bannissement dans TLS 1.3.

Exercice 2.2 — DES et Triple-DES

Travail demandé :

1. **DES-ECB et DES-CBC** : chiffrer/déchiffrer un texte de 128 octets en mode ECB puis CBC (IV aléatoire, padding PKCS7). Comparer les cryptogrammes.
2. **Visualiser la faiblesse ECB** : chiffrer une image 64×64 pixels en DES-ECB octet par octet ; reconstituer l'image chiffrée et observer que les motifs restent visibles.
3. **Triple-DES-CBC** : chiffrer le même message avec 3DES, clé 24 octets. Mesurer et comparer les temps de chiffrement DES vs 3DES sur 1 Mo.

Exercice 2.3 — AES : Advanced Encryption Standard

Travail demandé :

1. **Modes ECB/CBC/CTR** : chiffrer la même image avec AES-128-ECB, AES-256-CBC et AES-256-CTR. Afficher et commenter les trois images chiffrées (fuite de structure ECB visible).
2. **Avalanche en CBC** : modifier 1 bit du IV ; observer la propagation de l'erreur sur tous les blocs (visualiser le taux de bits différents bloc par bloc).
3. **Vulnérabilité nonce-reuse CTR** : chiffrer deux messages M1 et M2 avec le même nonce ; calculer $C1 \oplus C2 = M1 \oplus M2$ et récupérer partiellement les clairs.
4. Comparer AES-128 vs AES-192 vs AES-256 sur 10 Mo : mesurer le débit (Mo/s) et l'impact du nombre de tours sur les performances.

Exercice 2.4 — Les 5 Finalistes du Concours AES (NIST 1997–2000)

Contexte : Le NIST a évalué 15 candidats puis sélectionné 5 finalistes. Rijndael a été retenu comme AES en octobre 2000. Les 4 autres restent des alternatives solides.

Travail demandé :

1. **Étude architecturale** : pour chacun des 5 finalistes (Rijndael, Twofish, Serpent, RC6, MARS), décrire en 3 lignes la structure interne (SPN ou Feistel, taille de bloc, tours, originalités).
2. **Implémenter les 5 finalistes** (Rijndael, Twofish, Serpent, RC6, MARS) : utiliser des bibliothèques disponibles ; chiffrer le même message 128 bits avec les 5 algorithmes et comparer les cryptogrammes.
3. Benchmark comparatif : mesurer le temps de chiffrement et de déchiffrement de chaque finaliste sur 1 Mo de données aléatoires ; tracer un graphique en barres.
4. **Question** : l'algorithme *Serpent* a obtenu la meilleure note pour la sécurité mais n'a pas été retenu. Quel critère a fait pencher la balance vers Rijndael ?

TP 3 Cryptographie Asymétrique

Objectif : Implémenter Diffie-Hellman, RSA, ElGamal et ECC — sécurité et performances

Libs : cryptography · sympy · math

Exercice 3.1 — Échange de clés Diffie-Hellman (DH)

Travail demandé :

1. Implémenter DH : générer un grand premier p (au minimum 512 bits), choisir un générateur g , simuler l'échange complet entre A et B et donner K la clé partagée.
2. **Attaque Man-in-the-Middle** : simuler un attaquant qui intercepte les échanges entre A et B, substitue ses propres valeurs A' et B' , établit deux sessions distinctes. Illustrer par un schéma texte + valeurs et actions.
3. **Contre-mesure** : ajouter une signature ECDSA des clés publiques échangées pour authentifier l'échange et bloquer l'attaque MITM.

Exercice 3.2 — RSA : Rivest-Shamir-Adleman

Travail demandé :

1. **Implémenter RSA-512, 1024, 2048 (longueurs en bits) avec bibliothèque** : générer des paires pour chaque longueur, chiffrer une chaîne de 32 octets, déchiffrer et exporter les clés.
2. Chiffrement hybride RSA+AES : générer une clé AES-256 aléatoire, la chiffrer avec RSA, chiffrer un fichier de 1 Mo avec AES. Mesurer le temps total AES vs RSA.
3. **Question** : Pourquoi RSA ne peut pas chiffrer directement un message de taille arbitraire ? Qu'est-ce que le padding OAEP apporte par rapport à RSA textbook ?

Exercice 3.3 — Chiffrement ElGamal

Travail demandé :

1. Implémenter ElGamal : génération de clés (p premier > 512 bits, g générateur, x aléatoire, $y=g^x \bmod p$), chiffrement et déchiffrement d'un entier $M < p$.
2. Chiffrer $M = 12345$ et déchiffrer. Vérifier $D(E(M)) = M$. Observer que deux chiffrements du même M donnent des résultats différents (**non-déterminisme**).
3. **Malléabilité** : montrer que $E(M1) \cdot E(M2) = E(M1 \cdot M2 \bmod p)$: à partir de $C=(C1,C2)$, forger $E(2M) = (C1, 2C2 \bmod p)$ sans connaître M ni x .
4. Comparer les tailles de clés et de chiffrés : RSA-2048 (chiffré 256 octets) vs ElGamal-2048 (chiffré 512 octets, soit le double). Discuter les implications pratiques.

Exercice 3.4 (Supplémentaire) — Cryptographie sur Courbes Elliptiques (ECC)

Courbe : $y^2=x^3+ax+b \bmod p$ (Weierstrass). Loi de groupe : $P+Q$ via la corde et la tangente.
 ECDLP : $Q=kP$ difficile à inverser. ECC-256 $\approx$ RSA-3072 en sécurité (NIST SP 800-57).

Travail demandé :

1. Implémenter l'addition de points et la multiplication scalaire sur la courbe $y^2=x^3+7 \bmod 97$ (petits paramètres pédagogiques). Vérifier les propriétés du groupe.
2. **ECDH sur P-256** : avec Python-cryptography, générer les paires de A et B, calculer le secret partagé ECDH, dériver une clé AES-256 via SHA256.
3. Chiffrement hybride ECDH + AES : implémenter un système complet où A chiffre un message pour B en utilisant la clé publique de B (ECIES simplifié).

TP 4 Fonctions de Hachage Cryptographique

Objectif : Implémenter MD5, SHA-256, SHA-512 et analyser leurs propriétés de sécurité

Libs : hashlib (stdlib) · hmac · cryptography

Exercice 4.1 — MD5 (Message Digest 5)

Rappel : Sortie 128 bits. Construction Merkle-Damgård. 4 tours de 16 opérations (F, G, H, I). Collisions pratiques trouvées (Wang & Yu, 2004). Banni pour la sécurité, encore utilisé pour les checksums.

Travail demandé :

1. **Utilisation :** calculer le hash MD5 de 5 messages différents (chaîne vide, 1 octet, 1 Ko, 1 Mo, fichier binaire) avec hashlib.md5(). Vérifier que la sortie est toujours 128 bits.
2. **Effet avalanche :** modifier un seul bit de chaque message ; comparer les empreintes bit à bit et calculer le taux de bits différents (doit être $\approx 50\%$).

Exercice 4.2 — SHA-256 (Secure Hash Algorithm 2)

Rappel : Sortie 256 bits. Construction Merkle-Damgård avec compression basée sur des blocs de 512 bits et 64 constantes $K[i]$ = premiers 32 bits des racines cubiques de premiers. Standard : TLS, git, Bitcoin, JWT.

Travail demandé :

1. Implémenter SHA-256 (padding Merkle-Damgård, expansion des messages, 64 tours de compression) et valider contre hashlib.sha256() sur 10 vecteurs de test.
2. **Vérification d'intégrité :** simuler le téléchargement d'une archive Linux : calculer le SHA-256 du fichier local et le comparer au hash officiel publié — afficher OK / CORROMPU.

Exercice 4.3 — SHA-512 et comparaison générale

Rappel : SHA-512 : 512 bits, 80 tours, mots 64 bits — plus rapide que SHA-256 sur CPU 64 bits. SHA-3 (Keccak) : construction éponge — immune à la length extension par conception.

Travail demandé :

1. Calculer et comparer MD5, SHA-256, SHA-512 sur le même message. Afficher taille de sortie, temps de calcul et constater l'effet avalanche pour chacun.
2. Benchmark sur 100 Mo : mesurer le débit (Mo/s) de MD5, SHA-256, SHA-512 identifier le plus rapide et le plus lent sur votre machine.

TP 5 Signatures Numériques

Objectif : Générer et vérifier des signatures RSA, ElGamal et DSA — attaques et contre-mesures

Libs : cryptography · pycryptodome

Principe général : Signer : $S = \text{Sign}(\text{SK}, H(M))$ avec la clé privée. Vérifier : $\text{Verify}(\text{PK}, M, S) \in \{\text{Vrai}, \text{Faux}\}$ avec la clé publique. Garanties : authenticité, intégrité, non-répudiation.

Exercice 5.1 — Signature RSA (PKCS#1 v1.5 et PSS)

Exercice 5.2 — Signature ElGamal

Exercice 5.3 — DSA et ECDSA

TP 6 Application : Sécurisation des Communications

Objectif : Implémenter une application complète de communication sécurisée entre deux entités

Libs : socket · ssl · bluetooth (pybluez) · cryptography
· threading · qrcode

Contexte : A et B communiquent sur un canal non sécurisé. Le protocole doit garantir : confidentialité (chiffrement AES RSA), authenticité, intégrité et anonymat pour le vote électronique.

Exercice 6.1 — Sécurisation par Sockets TCP/IP

Exercice 6.2 — Sécurisation Bluetooth (RFCOMM)

Exercice 6.3 — Sécurisation sur Wi-Fi / UDP (application de chat)

Exercice 6.4 — Application de vote électronique sécurisé (utilisation de l'homomorphisme)